

# React js Interview Questions for Freshers



## React js Interview Questions for Freshers and Experienced

### React js Interview Questions: An Overview

React is an open-source, effective, and versatile [JavaScript framework](#) library that enables programmers to create straightforward, quick, and scalable online applications. It was made by Facebook employee Jordan Walke, a software developer. It was initially used in 2011 on Facebook. React makes it simple for developers with a background in Javascript to create online applications.

Learning React can be both exciting and challenging. There are so many topics to learn that it's difficult to know where to begin. Explore the world of React with a [React tutorial](#) to streamline your learning journey. That's why a [React roadmap](#) may be so useful. A roadmap provides an approach for learning React step by step so that you can enhance your skills. You can explore more concepts of React through [React JS Certification training](#). To put it simply, React Hooks are the functions that link the lifecycle characteristics from the function components with the React state. One of the things that is implemented is React Hooks.

## React js Interview Questions for Freshers

### 1. What is React?

A front-end, open-source Javascript package called React is helpful when creating user interfaces for single-page applications. Because it uses a component-based methodology, it is useful for creating intricate and reusable user interface (UI) components for online and mobile apps.

### 2. What are the advantages of using React and also its features?

## Advantages of React:

Model View Controller, or [MVC](#), is a commonly used acronym.

- **Utilizing Virtual DOM to Increase Efficiency:** React renders the view by using virtual DOM. Virtual DOM is, as its name implies, a virtual replica of the actual DOM. A new virtual DOM is built in a React application every time the data is modified. Rendering the user interface (UI) within the browser is far slower than creating a virtual DOM. Thus, the app's efficiency increases with the use of virtual DOM.
- **Mild learning curve:** Compared to frameworks like Angular, React has a mild learning curve. With React, anyone with minimal Javascript experience can begin developing web apps.
- **favorable to SEO:** With React, developers can create captivating user interfaces that are simple to use across a range of search

## Features of React:

- Rendering on the server is supported.
- Because real DOM manipulations are more expensive, it will use the virtual DOM instead of the real DOM (Data Object Model).
- It is in line with unidirectional data flow or data binding.
- It develops the view using reusable or modular user interface components.

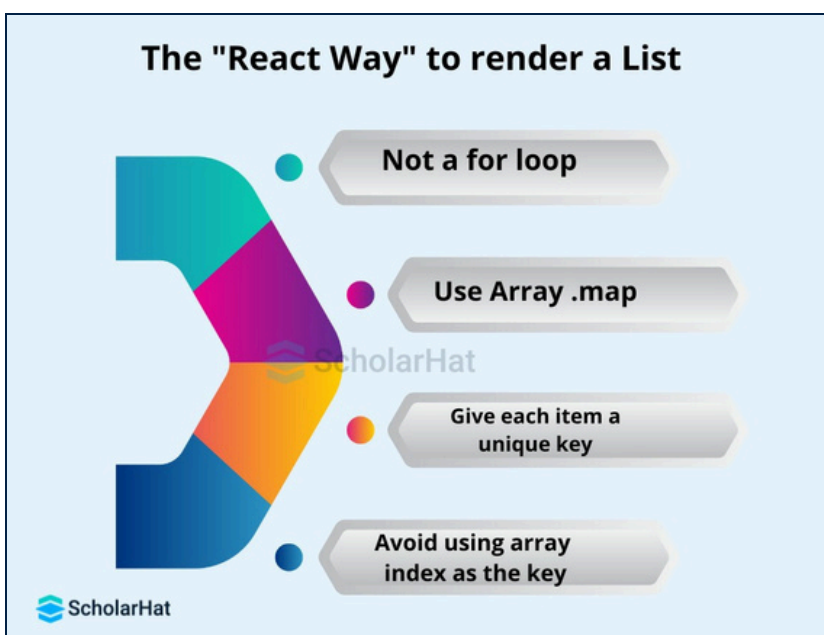
## 3. What are the limitations of React?

The following are a few of React's limitations:

- React is merely a library; it is not a full-fledged framework.
- It will take some time to completely understand all of React's features because there are so many of them.
- React may be hard for novice programmers to understand.
- Because JSX and inline templating will be used, the coding may get more complicated.

## 4. What are the keys in React?

When leveraging lists of elements, a key is a particular string attribute that has to be included.



Example :

```
const ids = [21,22,23,24,25]; const
listElements = ids.map((id)=>{ return(

    {id}
  )
})
}
```

Keys enable React to determine which elements were altered, added, or eliminated. To give each element in the array a distinct identity, keys should be assigned to them. React is unable to recognize the uniqueness or sequence of each element without keys. React uses keys to track which specific element was added, changed, and removed. Typically, keys are used to show a list of data obtained from an API.

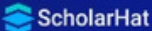
## 5. What is the virtual DOM? How does React use the virtual DOM to render the UI?

- Any web application requires DOM manipulation, but compared to other JavaScript operations, DOM manipulation is incredibly sluggish. Performing several DOM changes has an impact on the application's efficiency. Even when a small portion of the DOM changes, the majority of JavaScript frameworks update the entire document. Take a look at a list rendered inside the DOM, for instance. When an item in the list is updated or modified, the complete list is shown anew rather than just the modified or updated item. We refer to this as inefficient updating. The react team proposed
- the idea of virtual DOM to solve the issue of inefficient updating.

### Working of DOM:



# Upskill Your Frontend Stack with React 18



 Build End-to-end Project

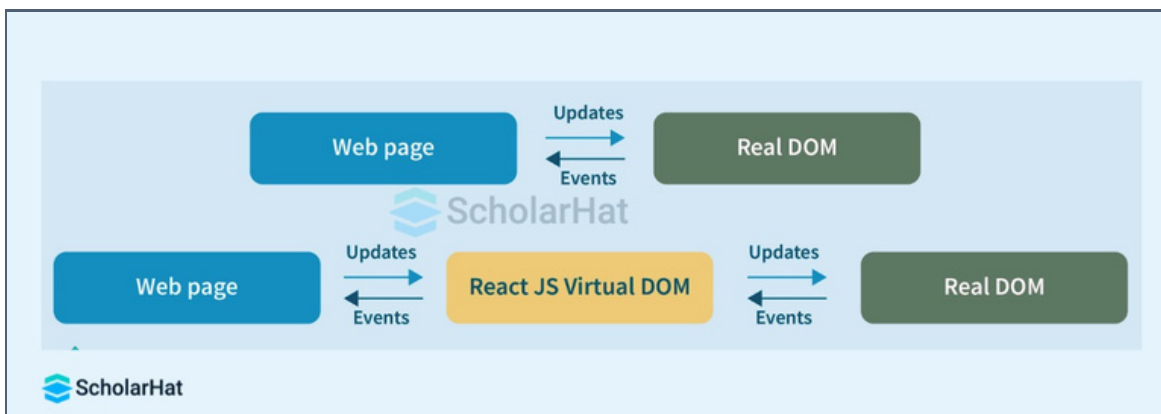
 Hands-On Labs to Build Confidence

 90% Hands-on Learning

 Interview Prep

 Get 40%-60% Salary Hike

[Book A Live Demo & Get 30% OFF](#)

Every DOM object has a virtual equivalent object (clone) with the same set of properties. The primary distinction between the virtual and real DOM objects is that modifications made to the former will not immediately appear on the screen. Think of a

virtual DOM object as the actual DOM object's blueprint. Every virtual DOM object is modified upon rendering of a JSX element. It's not inefficient to update every virtual DOM item, despite what one may believe. Since we are only changing the real DOM's blueprint, updating the virtual DOM happens considerably more quickly than updating the real DOM.

React builds the user interface using two virtual DOMs. The objects' present state is stored in one of them, and their former state is stored in the other. React examines the two virtual DOMs whenever the virtual DOM is changed to determine which virtual DOM objects were modified. React renders only those objects inside the real DOM after determining which items were modified, as opposed to rendering the entire real DOM. In this sense, React addresses the issue of inefficient updating by utilizing virtual DOM.

## 6. What are the differences between controlled and uncontrolled components?

The methods used to handle input from React elements are simply different for controlled and uncontrolled components. **A]**

**Controlled component:** In a controlled component, React controls the value of the input element. We maintain the state of the input element inside

the code, and by using event-based callbacks, any changes made to the input element will be reflected in the code as well.

The `onChange` method is called when a user enters data into a controlled component's input element. The code then determines if the entered value is valid or not. We update the state and re-render the input element with the updated value if the value is valid.

Example:

```
function
FormValidation(props) {
  let [inputValue, setInputValue] = useState("");
  let updateInput = e => {
    setInputValue(e.target.value);
  };
  return (
    <div>
      <form>
        <input type="text" value={inputValue} onChange={updateInput} />
      </form>
    </div>);
}
```

The code above illustrates how the status of the `inputValue` variable affects the value of the input element. The `updateInput` function handles modifications made to the input element.

### **B] Uncontrolled component:**

The DOM handles the input element's value in an uncontrolled component. Uncontrolled components' input elements function in the same way as standard HTML input form elements. The DOM manages the state of the input element. Event-based callbacks are not triggered when the value of the input element changes. React essentially does nothing in response to changes made to the input element.

The updated data is displayed immediately whenever the user inserts data in the input field. We can use `ref` to get the value of the input element.

## Example:

```
Function FormValidation(props) {  
  let inputValue= React.createRef();  
  let handleSubmit= e => {  
    alert(` Input value: ${inputValue.current.value}`); e.preventDefault();  
  };  
  return (  
    <div> <form onSubmit={handleSubmit}> <input type="text" ref={inputValue} />  
      <button type="submit">  
        Submit</button>  
    </form>  
  </div>  
  );  
}
```

The code above illustrates that we are not utilizing the onChange method to control the modifications performed to the input element. Rather, the value of the input element is accessed through the use of ref.

## 7. What are props in React?

The inputs of React components are called props. They can be named with a convention that resembles HTML-tag attributes, and they can be single-valued or objects with a set of values that will be supplied to React components during creation. Props can be defined as the information that is provided from a parent component to a child component.

Props' primary function is to offer various component functionalities, like:

- Passing custom data to the React component.
- Using through this.props.reactProp inside render() method of the component.
- Triggering state changes.

## 8. Explain the types of side effects in the React component.

- **Effects without Cleanup:** This side effect won't be utilized in useEffect since it doesn't prevent browser updates to the screen. It also enhances an application's responsiveness. Network queries, logging, manual DOM changes, etc. are a few typical examples.
- **Effects with Cleanup:** Following DOM updates, certain Hook effects will need to be cleaned up. For instance, memory must be cleaned up before setting up an external data source subscription to avoid the possibility of a memory leak. It is a known truth that React will do memory cleanup upon component unmounting. However, rather than only for one render() function, the effects will execute for every method. We can thus conclude that React will also clear up the effects from the previous render before the effects are executed the next time.

## 9. What are error boundaries, in short?

Error boundaries are a feature of React version 16 that lets us identify and fix render-phase issues. An error border is any component that makes use of one of the following lifecycle techniques. Inside a lifecycle method, the constructor's render phase error boundary is found.

## 10. What are React Hooks?

The built-in features known as React Hooks allow developers to use the state and lifecycle methods found in React components. These are brand-new features that are compatible with React 16.8. The three stages of a component's lifespan

are update, unmount, and mount. Components additionally have states and characteristics. Developers will be able to combine these techniques with hooks to enhance code reuse and increase component tree navigation flexibility.

You may access all of React's functionalities without constructing class components by using Hook. For instance, handling a component's state required a class component before React version 16.8. But now utilizing the useState hook, we can keep the state in a functional component.

## React js Interview Questions for Experienced

### 11. What is the use of Strict Mode in React?

Currently, StrictMode deals with the following problems:

- **Finding components with hazardous lifecycle methods:** In asynchronous react applications, it is not safe to use specific lifecycle methods. It becomes more challenging to make sure that specific lifecycle methods are not invoked when third-party libraries are used. If any class components employ risky lifecycle methods, StrictMode assists in alerting us.
- **Notice regarding the use of outdated string API:** It is advised to use callback refs rather than string refs for managing references if one is using an older version of React. If we are managing referees with string refs, StrictMode issues a warning. **Warning about the usage of findDOMNode:** Previously, the findDOMNode() method was used to search the tree of a DOM node.

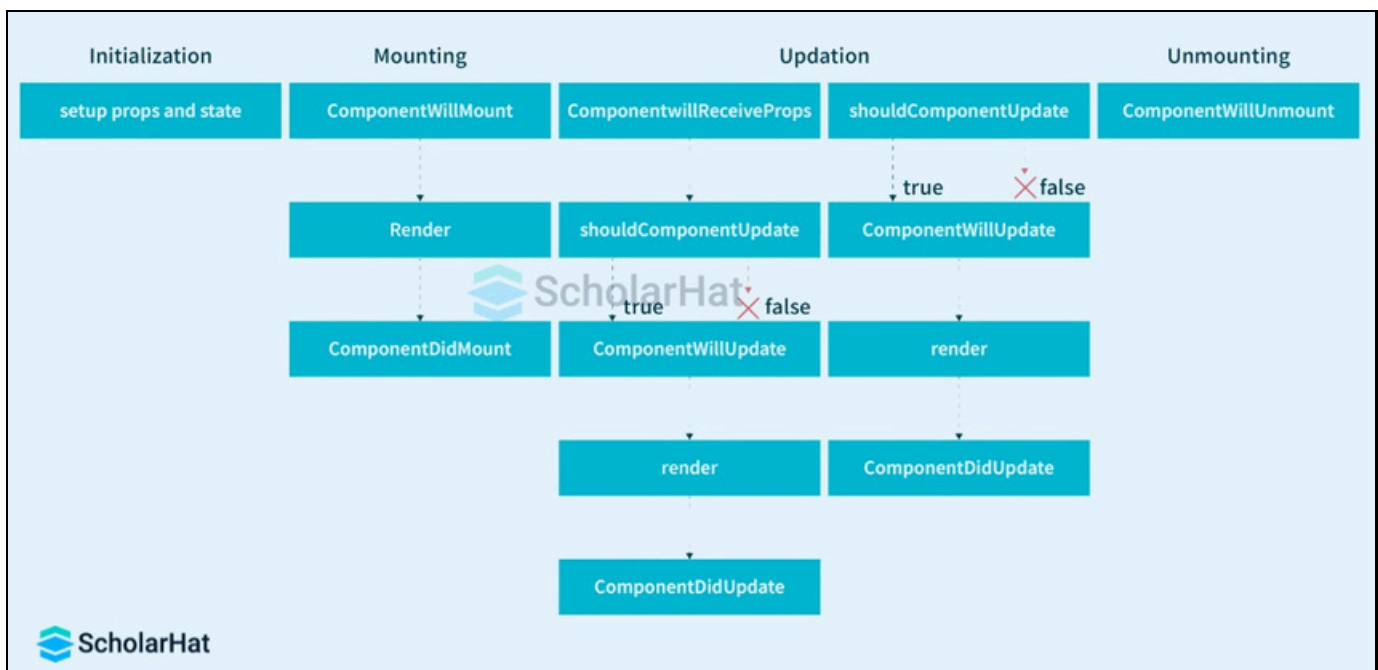
### 12. Name a few techniques to optimize React app performance.

- **using useMemo():** This React hook is used to cache CPU-intensive operations. Rendering in a React application may become sluggish at times when a CPU-expensive function is called repeatedly as a result of a component rendering again. Such routines can be cached using the useMemo() hook. The CPU-Expensive function is called only when necessary by utilizing useMemo().
- **Using React.PureComponent:** It is a base component class that determines whether a component needs to be changed by looking at its props and state. We can use React in place of the basic React.Component. Use PureComponent to cut down on needless component re-renders. **Maintaining State Colocation:** This involves relocating the state to the closest location that meets your needs. There are situations when a React application has a large number of pointless states inside the parent component, which makes the code more difficult to read and manage. Not to mention, having a lot of states inside one component causes the component to needlessly re-render. It is preferable to move states to a different component that is less important to the parent component. **Lazy Loading** - It is a technique used to minimize the load time of a React app. To minimize the danger of web app performance, lazy loading is used.

### 13. What are the different phases of the component lifecycle?

The lifecycle of a React component consists of four stages. They are as follows:

- **Mounting:** Inserting the elements into the browser's DOM is referred to as mounting. The complete browser DOM that has already been rendered would not be refreshed because React uses VirtualDOM. The lifecycle methods componentWillMount and componentDidMount are included in this phase. **Updating:** Whenever a component's state or props change, an update is made during this phase. Lifecycle methods like render, shouldComponentUpdate, componentDidUpdate, and componentWillUpdate will be available in this phase.
- **Unmounting:** The component will be eliminated from the DOM or unmounted from the browser DOM during this final stage of the component lifecycle. The componentWillUnmount lifecycle function will be used in this phase.



## 14. What are the lifecycle methods of React?

React lifecycle hooks provide you excellent control over what occurs at the invoked point by including methods that are automatically called at various stages of the component lifecycle. It gives you the ability to efficiently direct and influence events that occur during the component lifecycle. Assume simply these two: if you are constructing the YouTube application, for instance, the program will require a network to buffer the videos and it will utilize battery power. If the user opens another program after the video has finished playing, you should ensure that the battery and network are being used as effectively as possible.

**The several lifespan techniques are:**

- **constructor()**: When a component is started without any further action taken, this function will be called. Establishing the initial values and state is helpful.
- **getDerivedStateFromProps()**: This function will be triggered immediately before the DOM element(s) are displayed. Setting up the state object based on the initial props is helpful. The function `getDerivedStateFromProps()` takes a state as an input and returns an object that has modified the state. When a component has to be updated, this function will be called first.
- **render()**: This function outputs, or re-renders, the HTML to the DOM with the updated modifications. While the other methods are optional and will only be invoked if they are defined, the `render()` method is mandatory and will always be called.
- **componentDidMount()**: This function will be triggered following the component's rendering. You can execute commands that need the component to already be stored in the DOM by using this technique.
- **shouldComponentUpdate()**: This method will return a Boolean result that will indicate whether React should continue rendering or not. This method's default value is going to be `True`.
- **getSnapshotBeforeUpdate()**: This function will give you access to the state before the update as well as the props. It is conceivable.
- 

## 15. Does React Hook work with static typing?

The practice of checking code during compilation to make sure all variables will be statically typed is known as static typing. Functions called React Hooks are made to ensure that every attribute is statically typed. We can utilize custom Hooks in the

React API to enforce stricter static typing within our code.

## 16. How does the performance of using Hooks differ in comparison with the classes?

Many of the overheads associated with classes, such as instance creation and event binding, can be avoided by using React Hooks.

React hooks produce smaller component trees because they prevent the nesting that occurs in HOCs (Higher Order Components) and because they render props, React has to work less.

## 17. Do Hooks cover all the functionalities provided by the classes?

We aim to have all class functionalities covered by Hooks as soon as possible. The following methods are not yet included in Hooks and do not have Hook equivalents:

- `getDerivedStateFromError()`,
- `getSnapshotBeforeUpdate()`,
- `componentDidCatch()`

A few third-party libraries might not be compatible with Hooks now because it's still early in the development stage, but they will be added soon.

## 18. What are Higher Order Components?

We may create components for React applications that are remarkably similar to one another with very subtle differences. Creating identical components might not always be a problem, but when creating larger apps, we need to maintain DRY code. For this reason, we need an abstraction that enables us to create this logic once and share it with other components. We can make that abstraction thanks to HOC.

Example:

**Initialization:** In this stage, the React component gets ready for the challenging road ahead by setting up the default props and starting state.

The following component displays the users' list:

```
// "GlobalDataSource" is some global data source
class UsersList extends React.Component {
  constructor(props) {
    super(props);
    this.handleChange = this.handleChange.bind(this);
    this.state = {
      users: GlobalDataSource.getUsers(),
    };
  }

  componentDidMount() {
    { // Listens to the changes added
      GlobalDataSource.addChangeListener(this.handleChange);
    }
  }

  componentWillUnmount() {
    { // Listens to the changes removed
      GlobalDataSource.removeChangeListener(this.handleChange);
    }
  }
}
```

```

}
handleChange() {
  // States gets Update whenever data source changes
  this.setState({
    users: GlobalDataSource.getUsers(),
  });
}
render() {
  return ( <div>
    this.state.users.map((user) => (
      <UserData user={user} key={user.id} />
    )
  )
  </div>);
}
}
}
}

```

These elements call distinct methods to the same API endpoint, they both have comparable functionality. We are aware that the function HOC accepts a component as input and outputs another component. A function named HOC has been established in the code above. It returns a component and carries out operations that are common to the UsersList and ArticlesList components. The function that invokes the API endpoint method is the second parameter in the HOC function. The componentDidMount and componentWillMount methods now include less redundant code.

Now that we understand the idea of Higher-Order Components, we can render the UsersList and ArticlesList components as follows:

```

const ArticlesListWithHOC = HOC(ArticlesList, (GlobalDataSource) => GlobalDataSource.getArticles());
const UsersListWithHOC = HOC(UsersList, (GlobalDataSource) => GlobalDataSource.getUsers());

```

Keep in mind that our goal is to use HOC to share a single functionality among several components, not to alter the functionality of each component individually.

## 19. How often does the React useState update? Why?

React updates change slowly because developers create queues using useState to improve performance. Applicants should be aware that useState updates the state object asynchronously rather than directly implementing changes to it.

## 20. What are Error Boundaries?

Error boundaries are a feature of React version 16 that lets us identify and fix render-phase issues. An error border is any component that makes use of one of the following lifecycle techniques. An error boundary can identify errors inside a lifecycle method, the constructor's render phase.

## 21. What is an Event in ReactJS?

Events are the things that a computer or user performs. In ReactJS, events are things like clicking and hitting keys. Events in React are written in CamelCase, as opposed to lowercase as in HTML.

## 22. What is Redux?

One widely used library for front-end programming is called Redux. It has a ton of capabilities that make it easier to use UI components to do complex front-end jobs. Redux applications run smoothly across a variety of framework contexts and are simple to test.

## 23. What are some advantages of using Flux?

There are many benefits associated with flux, the primary ones being:

- By guaranteeing consistency in the way code is executed, it helps to avoid runtime mistakes.
- With unidirectional data flow, updates can be carried out more successfully, increasing the application's efficiency.

## 24. Can you list the main differences between React and React Native?

- **1. Platform:** React is a library for building user interfaces in web applications, whereas React Native is a framework for building native mobile applications for iOS, Android, and other platforms.
- **2. Language:** React uses JavaScript, while React Native uses a combination of JavaScript and a subset of CSS called Flexbox for styling.
- **Components:** Both React and React Native use components as the building blocks of user interfaces, but the components in React are designed for web browsers, while the components in React Native are designed for mobile devices and can include native UI elements such as View, Text, Image, etc
- **3. Styling:** React uses CSS for styling, while React Native uses a subset of CSS called Flexbox for styling, along with some additional styling options specific to mobile applications.
- **4. UI/UX:** React is designed to create web-based user interfaces, while React Native is designed to create mobile user interfaces that look and feel like native applications, providing a more native-like user experience.
- **5. Performance:** React Native uses native UI components, which are rendered directly by the device's GPU. This makes it more efficient in terms of performance compared to React, which relies on a virtual DOM to update the UI.
- **6. Platform-specific APIs:** React Native provides access to native APIs of the underlying mobile platform, such as the camera, geolocation, etc., allowing developers to create more sophisticated and feature-rich mobile applications compared to React, which primarily focuses on web-based UI.
- **7. Development workflow:** React Native requires a different development setup compared to React, as it involves building, compiling, and running native code on mobile devices, whereas React can be developed and run in a web browser.
- **8. Code Reusability:** One of the main advantages of React Native is its ability to reuse a significant portion of code between different platforms (iOS, Android, etc.), which can result in faster development and easier maintenance. Conversely, React components are typically designed specifically for web browsers and may not be easily reusable on other platforms.
- **9. Ecosystem:** While React has a mature ecosystem with a large number of libraries, tools, and resources available for web development, React Native has its own ecosystem tailored for mobile app development, which includes libraries, components, and tools specific to mobile platforms.
- 

These are some of the key differences between React and React Native. While they share similarities, they are designed for different platforms and have their strengths and use cases. Developers should choose the one that best fits their specific requirements and target platform.

## 25. Why do we need to transpile React code?

React code is written in the JSX, but unfortunately, no browser can execute JSX directly as they are built to read only regular JavaScript.

Hence we are required to use tools like Babel to transpile JSX to JavaScript so that the browser can execute it.

## 26. What are the most common approaches for styling a React application?

- **CSS Classes:**

React allows you to assign class names to components, just as class names are assigned to a DOM element in HTML. When developers start using React after developing traditional web applications, they often choose CSS classes because they already know it.

- **Inline CSS**

Style React to elements with inline CSS so styles can be completely limited to the element. However, certain style properties are not available for inline styles. For example, the style of pseudo-classes like hover.

- **Preprocessors (Sass, Stylus, and Less)**

Preprocessors are often used in React projects. This is because, like CSS, they are easy for developers to understand and are often already used when React is integrated into a legacy application.

- **CSS-in-JS modules (styled-components, emote, and styled JSX)**

CSS-in-JS modules are popular for designing React applications because they integrate tightly with React components. With them, for example, styles can be changed at runtime based on a React property. Also, most of these systems assign all styles to the corresponding styleable component by default.

## 27. What are some of the performance optimization strategies for React?

- **Using UseMemo:** useMemo is a React hook used to cache CPU expensive operations. A CPU-intensive function that is called multiple times due to component representation can cause slow rendering. The UseMemo hook can be used to save such operations. Using UseMemo, the CPU Expensive function is called only when needed.
- **Use callback:** It can be used to achieve a similar result as the result of using UseMemo. **Lazy loading:** Lazy loading is a technique used to reduce the loading time of a React application. It helps minimize performance risk in web applications by loading components as the user moves through the application.

## 28. What is prop drilling and how to avoid it?

- Sometimes when developing React JS applications, it is necessary to pass data from a component higher in the hierarchy to a deeply nested component. To pass data between such components, we pass a property to the parent component and keep passing it to the next component in the hierarchy until we reach a deeply nested component.
- The downside of prop drilling is that the data is accessed by components that should not otherwise be aware of the data, which also makes the code harder to maintain.
- Custom drilling can be avoided using the context API or some state management library.

## 29. What is the StrictMode component and why would you use it?

- `<StrictMode />` It is a component included with React to provide additional visibility of potential issues in components. Let's Suppose the application is running in development mode. In this case, any issues are logged to the development console, these warnings are not shown if the application is running in production mode.
- Programmers use `<StrictMode />` to find problems like deprecated lifecycle methods and legacy patterns, to ensure that all React components follow current best practices.

- `<StrictMode />` can be applied at any level of an application component hierarchy, it allows it to be adopted incrementally within a codebase.

## 30. What's the difference between Redux and MobX state managers? Highlight the pros and cons of each.

Both Redux and MobX are popular state managers used in JavaScript-based applications, but their approaches to managing state and updating user interfaces are different.

### A] Redux:

Redux is a predictable state container for JavaScript applications. Redux follows a unidirectional data flow model, which means that data only moves in one direction in the application. Redux stores all application states in a single repository managed by reducers. Reducers are functions that take the current state and return a new state to the function.

#### Pros:

- Predictability: Redux provides a predictable state management model that makes it easy to infer how data in an application changes over time.
- Debugging: Redux makes it easy to debug the application state by keeping a log of all actions sent to the store.
- Community Support: Redux has a large community and many resources available, making it easy to find help when you need it.

#### Cons:

- Boilerplate code: Redux requires a lot of general code to start writing, which can be time-consuming and cumbersome for small projects.
- Steep learning curve: Redux has a steep learning curve, especially for developers new to the concept of state management.
- Verb: Redux can be verbose in some cases, which can make the code difficult to read and understand.

### B] MobX:

MobX is a simple and scalable state management library for JavaScript applications. MobX uses observations to track state changes and automatically updates the UI when the state changes. Observables are objects whose changes can be monitored, and when they change, all components that depend on them are automatically updated.

#### Pros:

- Ease of Use: MobX is very easy to use and requires minimal installation and setup, making it a good choice for small to medium projects.
- Performance: MobX is very fast and efficient because it updates the UI only when needed and minimizes unnecessary rendering.
- code simplicity: MobX reduces the amount of code needed to manage space, making the code simpler and easier to read.

#### Cons:

- Low predictability: The reactive nature of MobX can make it difficult to derive application state changes, especially in complex applications.
- Limited Community: MobX has a smaller community compared to Redux, which can make it difficult to find help and resources.

- Magic: Due to its reactive nature, MobX can sometimes feel like magic, which can make it difficult to understand what's going on under the hood.

### 31. What is ReactJS used for?

React Js is used to create User Interfaces (UI) which enhance the speed of the program.

### 32. List 6 essential skills for React JS developer.

While learning React, major organizations view React JS skills as a priority. React developers should be hungry to level up their React JS skills for crafting incredibly responsive web applications.

#### React JS skills:

- HTML + CSS
- JSX
- JavaScript Fundamentals + ES6
- Git
- Node + npm
- Redux

### 33.Explain an event in React.js?

React.js records these activities as events when a user hits a key, clicks the mouse, or does any other action on the computer, or when the machine initiates an action.

- Instead of using HTML's lowercase for event names, React.js uses camelCase.
- Because of JSX, React.js passes a function as an event handler rather than a string as HTML does.

### 34.How do Lists work in React.js?

The making of lists in React.js is comparable to that of lists in standard Javascript. Lists help to show menus on websites and for displaying data in a structured way. You use the map() method to traverse lists.

As an illustration,

- The map() function takes an array of numbers and multiplies their value by 2 var numbers = [2,4,6,8,10].
- numbers are returned by const multiplyNums.map((number => "return (number\*2)););
- multiplyNums in console.log;
- Results: The console will record the Javascript output. In the example above, the output is [4, 8, 12, 16, 20].

### 35.What is a synthetic event in ReactJS?

In React JS, a Synthetic event is an object that serves as a cross-browser wrapper of the browser's native event.

### 36.Can you tell two downsides of React?

In React, It is difficult to integrate with an MVC framework like Rails. Also, users need to know about integrating user interfaces into the MVC framework.

### 37.What does super keyword mean in React js?

It is used to call super or parent class.

### 38. What sort of tools might you use in the build steps to optimize the compiled output React code?

In React, there are specific build tool plugins, such as the babel-react-optimize presets that involve compiling code into a format that optimizes React, such as automatically compiling any `React.createElement()` calls into a JavaScript Object that inlines right into the source code:

```
class MyComponent extends React.Component {
  render() {
    (
      <div className={this.props.className}>
        <span>Hello World</span>
      </div>
    );
  }
}
```

becomes

```
class MyComponent extends React.Component {
  render() {
    return (
      _jsx('div', { className: this.props.className }, void 0,
        _jsx('span', {}, void 0, 'Hello World')
      )
    );
  }
}
```

### 39. Explain the term stateless components

In React Js, Stateless components are pure functions that render DOM based solely on the properties provided to them.

### 40. What is React Router?

It is a routing library that allows you to add new screen flows to your application, and it also keeps the URL in sync with what's being shown on the page. Summary: You can prepare for interviews by reading this post, which contains the most common interview questions and

answers for ReactJS and [React Hooks](#). Also for better performance, you can do our [React JS Certification training](#).

Additionally, keep in mind that your performance throughout the interview will depend on more than just your technical abilities;

## FAQs

### Q1. What are the popular animation package in React ecosystem?

Popular animation packages in the React ecosystem are as follows:

- React Motion
- React Transition Group

### Q2. What is Jest?

It is a JavaScript unit testing framework created by Facebook based on Jasmine. It offers automated mock creation and a js dom environment. It is also used as a testing component.

### Q3. What is dispatcher?

It is a central hub of the app where you will receive actions and broadcast payload to registered callbacks.

### Q4. What is meant by callback function? What is its purpose?

The callback function should be called when setState has finished, and the component is re-rendered. The setState is asynchronous, which is why it takes in a second callback function.

### Q5. Explain the term high order component

It is also known as HOC is an advanced technique for reusing component logic. It is not a part of the React [API](#), but they are a pattern that emerges from React's compositional nature.

### Q6. Explain the Presentational segment

A presentational part is a segment which allows you to renders HTML. The segment's capacity is presentational in markup.

### Q7. Explain yield catchphrase in JavaScript

The yield catchphrase is utilized to delay and [resume](#) generator work, which is known as yield catchphrase.